

Prueba Técnica – Datos SECOP

Analítico/a de Ingeniería y Ciencia Analítica Cumplimiento

Juan Camilo García Aguirre – Talento B

GCIA INGENIERIA Y CIENCIA ANALITICA CUMP

Tabla de contenido.

Entendimiento del DataSet.	2
Extracción de la información.	2
Exploración y análisis.	4
Transformación y limpieza.	6
Creación de banderas de apoyo.	8
Creación del quality_log.	8
Separación de datos históricos vs financieros.	9
Normalización.	11
Código SQL.	12
Análisis descriptivo.	12
Conclusiones del análisis descriptivo histórico.	13
Conclusiones del análisis descriptivo financiero.	13
Anomalías año 2025.	14
Conclusiones.	15

Entendimiento del DataSet.

El primer paso que realicé fue entrar al portal de **SECOP Integrado - Datos Abiertos Colombia** para conocer de primera mano la información disponible. Antes de escribir código o pensar en análisis, me enfoqué en explorar la página, revisar los conjuntos de datos publicados y entender qué tipo de información se encontraba allí.

A partir de esta exploración inicial, comprendí qué es el **SECOP**, identificándolo como el sistema donde se registra la información de los procesos de contratación pública en Colombia.

Luego, revisé con calma el contenido del dataset: observé las columnas, los nombres de los campos, los tipos de datos y la información que representaba cada uno. Este ejercicio me ayudó a hacerme una idea clara de cómo estaban organizados los datos y qué tan completos o consistentes eran, incluso antes de descargarlos y trabajarlos en Python.

Quiero recalcar que **el entendimiento de los datos fue uno de los pasos más importantes de todo el ejercicio**. Tomarme el tiempo para conocer el origen y el significado de la información me permitió tratar los datos de forma correcta en las siguientes etapas.

Extracción de la información.

Después de haber entendido el dataset, el siguiente paso fue la extracción de la información. Gracias a la exploración previa del portal de SECO, encontré que la plataforma cuenta con una API que permite consumir y descargar los datos de forma directa desde herramientas externas, como Python.

Para poder usarla correctamente, leí el manual de usuario y así logré entender cómo funcionaban los parámetros de consulta, los filtros y la forma en que se entregaban los datos.

¿Qué hay en este conjunto de datos?

Filas	Columnas	Cada fila es un
20,7M	22	Proceso

En este punto me enfrenté a la primera gran decisión del desarrollo. El dataset completo está conformado por más de 20 millones de registros, una cantidad bastante grande para trabajarla de forma local. Consideré que manejar todo ese volumen de datos iba a hacer que los procesos posteriores fueran mucho más lentos y complejos, especialmente teniendo en cuenta que debía realizar validaciones, análisis y procesos iterativos sobre la información.

Por esta razón, tomé la decisión de descargar únicamente la información correspondiente al año 2025. La prueba técnica no exige un número mínimo de registros ni analizar varios años, y además solicita específicamente la identificación de anomalías para el año 2025. Al centrarme solo en este período, logré reducir significativamente el volumen de datos, optimizar los tiempos de procesamiento y enfocar el análisis directamente en el objetivo planteado.

Antes de descargar los datos, **procedí a crear el proyecto en Python**, definiendo desde el inicio una estructura clara que me permitiera trabajar de forma ordenada. Para esto, creé un entorno virtual (venv) y organicé el proyecto con la siguiente distribución de carpetas:



La carpeta **src** fue pensada para contener todos los archivos .py con funciones que me apoyaran en la **extracción, limpieza, transformación y almacenamiento** de la información. Por su parte, la carpeta **notebooks** la utilicé para trabajar con archivos .ipynb, los cuales me permitieron explorar los datos, transformarlos y limpiar la información de manera más visual y progresiva.

Para realizar la descarga desde Python, **creé una función específica para consumir la API**, incluyendo los filtros por año y utilizando el concepto de **paginación**. Esto fue necesario para evitar sobrecargar la API o provocar fallos debido a la gran cantidad de información solicitada en cada consulta.

```
DATASET_ID = "rpmr-utcd"
DOMAIN = "www.datos.gov.co"
LIMIT = 10000

def descargarDatosSecop(fecha_inicio="2025-01-01T00:00:00", fecha_fin="2025-12-31T23:59:59"):
    client = Socrata(DOMAIN, None, timeout=60)
    offset = 0
    resultados = []

    while True:
        print(f"Descargando registros desde offset {offset}...")

        try:
            registros = client.get(
                DATASET_ID,
                limit=LIMIT,
                offset=offset,
                where=(
                    f"fecha_de_firma_del_contrato between "
                    f"'{fecha_inicio}' and '{fecha_fin}'"
                )
            )

        except Exception as e:
            print(f"⚠ Error temporal en offset {offset}: {e}")
            print(f"🔄 Reintentando en 5 segundos...")
            time.sleep(5)
            continue

        if not registros:
            break

        resultados.extend(registros)
        offset += LIMIT

    return pd.DataFrame.from_records(resultados)
```

Una vez realizada la descarga, obtuve un total de 1.679.887 registros, los cuales fueron cargados y almacenados en un DataFrame para su posterior procesamiento.

```
dataSecop = descargarDatosSecop()
dataSecop.shape
```

Posteriormente, decidí cargar la información cruda en la base de datos (PostgreSQL) específicamente en el esquema “datoscrudos”. Además, se definieron varios esquemas, cada uno asociado a una fase diferente del procesamiento de la información.

```
motor = getEngine()
cargarDatosCrudosSecop(dataSecop, motor, "secop_contratos_2025")
```

Esta decisión se tomó con el fin de mantener trazabilidad sobre los datos originales, permitiendo conservar una copia sin transformaciones ni validaciones aplicadas. De esta forma, es posible auditar el proceso completo, comparar los datos en cada etapa y garantizar transparencia sobre cómo se fue transformando la información a lo largo del ejercicio.



Schemas

“Más adelante se explicará cual es el objetivo y que contiene cada esquema en específico”

```
CREATE SCHEMA IF NOT EXISTS analisisfinanciero;
```

```
CREATE SCHEMA IF NOT EXISTS analisisishistorico;
```

```
CREATE SCHEMA IF NOT EXISTS datoscrudos;
```

```
CREATE SCHEMA IF NOT EXISTS procesados;
```

“Código SQL para crear los esquemas necesarios”

Exploración y análisis.

Durante este proceso identifiqué varios aspectos en los datos que requerían ajustes antes. Estos hallazgos fueron clave para entender mejor la naturaleza de la información y definir cómo debía tratarse cada caso.

- Encontré que algunas columnas de tipo texto, como nivel_entidad, departamento_entidad, municipio_entidad, objeto_del_proceso, objeto_a_contratar y nom_raz_social_contratista, contenían caracteres especiales o símbolos que no deberían hacer parte del contenido normal de estos campos, tales como paréntesis, signos de admiración o signos de interrogación. Estos elementos podían afectar la consistencia de la información y dificultar posteriores procesos de análisis o agrupación.

nombre_entidad text
(Secretaría Distrital de Integración Social)
.SECRETARÍA DISTRITAL DE SALUD..
ACI Medellín
ACUEDUCTO Y ALCANTARILLADO DE POPAYAN S.A. E.S.P.**
ADMINISTRACIÓN PÚBLICA COOPERATIVA DEL MUNICIPIO DE ENCINO AGUASAN A.P.C. E.S.P
ADMINISTRADORA COLOMBIANA DE PENSIONES COLPENSIONES
ANDES

nombre_proveedor text
LEIDY ESPERANZA GUACA...
-
-ANGELICA VIRGINIA REYES H...
-FUNDAPU
-FUNDAPU
ILIANA MARGARITA GARZON ...
#NARRARELFUTURO
&NTELIGENTE S.A.S
*CARLOS EDWARD VALENCIA ...
*DIVERSIONES DEL OCCIDEN...
*LUZ VIVIANA SOTO SOTO
*TECHNICAL ADJUSTMENTS ...
*
*

- La columna documento_proveedor se identificaron valores que incluían caracteres no esperados como comillas, paréntesis o signos especiales, los cuales no corresponden al formato habitual de este tipo de identificadores y requerían ser depurados.

documento text
52905716
34983846
? ? ? ? ?
53013056
1128627468

- La existencia de categorías duplicadas debido al uso inconsistente de mayúsculas y minúsculas. Por ejemplo, valores como “ejecución” y “EJECUCIÓN” representaban la misma categoría, pero eran interpretados como registros distintos, lo que podía generar errores en conteos y análisis agregados.
- En cuanto a las fechas, se detectaron inconsistencias temporales, como fechas de inicio anteriores a la fecha de firma del contrato o fechas de finalización anteriores a la fecha de inicio. Este tipo de situaciones evidenció la necesidad de aplicar reglas de validación para garantizar la coherencia cronológica de los contratos.
- Se identificaron contratos con valores iguales a cero. Al analizar estos casos, descubrí que estaban asociados principalmente al tipo de contrato “LICITACIÓN PÚBLICA ACUERDO MARCO DE PRECIOS”, el cual permite este comportamiento, por lo que este tipo de registros no debía ser considerado como un error, sino como una particularidad válida del modelo de contratación.

modalidad_de_contratación text	valor_contrato bigint
LICITACIÓN PÚBLICA ACUERDO MARCO DE PRECIOS	0
LICITACIÓN PÚBLICA ACUERDO MARCO DE PRECIOS	0
LICITACIÓN PÚBLICA ACUERDO MARCO DE PRECIOS	0
LICITACIÓN PÚBLICA ACUERDO MARCO DE PRECIOS	0
LICITACIÓN PÚBLICA ACUERDO MARCO DE PRECIOS	0
LICITACIÓN PÚBLICA ACUERDO MARCO DE PRECIOS	0
LICITACIÓN PÚBLICA ACUERDO MARCO DE PRECIOS	0
LICITACIÓN PÚBLICA ACUERDO MARCO DE PRECIOS	0

- Observé que un mismo contrato podía aparecer en múltiples registros, debido a que su estado cambia a lo largo del tiempo. Este comportamiento resultó especialmente importante para el análisis, ya que refleja la evolución del contrato y debía ser tenido en cuenta para evitar interpretaciones incorrectas.
- Se encontraron fechas de inicio nulas en algunos registros. Al profundizar en estos casos, entendí que dichas fechas estaban relacionadas con determinados estados del contrato que aún no han iniciado su ejecución, por lo que la ausencia de la fecha no representaba un error, sino una condición válida dentro del ciclo de vida del contrato.
- Se encontraron datos duplicados en los registros

Transformación y limpieza.

Una vez que la información cruda fue almacenada en la base de datos, procedí a recuperarla desde allí y cargarla en un nuevo DataFrame, con el objetivo de iniciar la fase de transformación y limpieza de los datos.

```
motor = getEngine()

dataCruda = pd.read_sql(
    "SELECT * FROM datoscrudos.secop_contratos_2025",
    con=engine
)

dataCruda.shape
```

En este contexto, la limpieza de datos hace referencia a corregir, estandarizar y ajustar la información para que sea consistente, legible y adecuada para el análisis. No se trata de eliminar información sin criterio, sino de asegurar que los datos representen correctamente la realidad que describen.

El primer paso fue identificar el tipo de dato de cada columna, revisando cuáles eran realmente textos, fechas o valores numéricos, y evaluando si algunos de ellos debían ser ajustados para un mejor manejo. Esto permitió definir los tipos de datos más adecuados y prevenir errores en cálculos, comparaciones o validaciones posteriores.

Posteriormente, creé funciones específicas para la limpieza de datos, tanto para columnas de texto como para columnas numéricas. Estas funciones se apoyaron en los hallazgos previos y permitieron corregir caracteres no deseados que aparecían en varias columnas, como símbolos especiales o formatos inconsistentes, los cuales podían afectar la calidad y la interpretación de la información.

Adicionalmente, realicé un proceso de estandarización de mayúsculas y minúsculas en ciertas columnas categóricas. Este paso fue fundamental para unificar valores que representaban la misma categoría, pero que se encontraban escritos de forma diferente, por ejemplo, por el uso de mayúsculas o minúsculas. De esta manera, se evitó que una misma categoría fuera tratada como si fueran varias distintas.

```
columnasDateTime = ['fecha_de_firma_del_contrato', 'fecha_inicio_ejecuci_n', 'fecha_fin_ejecuci_n']
columnasNumeric = ['valor_contrato']
normalizacionTexto = ['estado_del_proceso', 'modalidad_de_contrataci_n', 'nivel_entidad', 'tipo_de_contrato', 'origen', 'nom_raz_social_contratista', 'tipo_documento_proveedor' ]
strings = ['nivel_entidad', 'departamento_entidad', 'municipio_entidad', 'objeto_del_proceso', 'nom_raz_social_contratista', 'objeto_a_contratar', 'objeto_del_proceso']

for col in columnasDateTime:
    dataCruda[col] = pd.to_datetime(dataCruda[col], errors='coerce')

for col in columnasNumeric:
    dataCruda[col] = pd.to_numeric(dataCruda[col], errors='coerce')

for col in normalizacionTexto:
    dataCruda[col] = dataCruda[col].astype(str).str.strip().str.upper()

for col in strings:
    dataCruda[col] = dataCruda[col].astype(str).apply(limpiezaStrings)

dataCruda['nombre_de_la_entidad'] = dataCruda['nombre_de_la_entidad'].astype(str).apply(limpiezaStrings).str.upper()
dataCruda['documento_proveedor'] = dataCruda['documento_proveedor'].apply(limpiarDocumento)
```

Después de realizar este proceso, procedí a verificar que los datos hubieran quedado correctamente ajustados. Revisé que los tipos de datos fueran los correctos, que no persistieran caracteres no deseados y que las categorías estuvieran correctamente unificadas

```
dataCruda['estado_del_proceso'].value_counts(dropna=False)
```

estado_del_proceso	# count
EN EJECUCIÓN	462098
MODIFICADO	438111
ACTIVO	329835
CELEBRADO	180437
APROBADO	136739
TERMINADO	68988
CERRADO	39839
SUSPENDIDO	11438
CEDIDO	10034
LIQUIDADO	1634

```
dataCruda['modalidad_de_contrataci_n'].value_counts(dropna=False)
```

modalidad_de_contrataci_n	# count
CONTRATACIÓN DIRECTA	1070040
CONTRATACIÓN RÉGIMEN ESPECIAL	285745
CONTRATACIÓN DIRECTA (LEY 1150 DE 2007)	125753
MÍNIMA CUANTÍA	68437
RÉGIMEN ESPECIAL	31607
SELECCIÓN ABREVIADA DE MENOR CUANTÍA	16684
CONTRATACIÓN MÍNIMA CUANTÍA	16658
CONTRATACIÓN DIRECTA (CON OFERTAS)	15431
SELECCIÓN ABREVIADA SUBASTA INVERSA	13647
CONTRATACIÓN RÉGIMEN ESPECIAL (CON OFERTAS)	13077

```
dataCruda['origen'].value_counts(dropna=False)
```

origen	# count
SECOPII	1497097
SECOPI	182790


```
dataCruda['nivel_entidad'].value_counts(dropna=False)
```

nivel_entidad	# count
TERRITORIAL	1224876
NACIONAL	432354
CORPORACIÓN AUTÓNOMA	18770
NO DEFINIDO	3887

```
dataCruda['tipo_de_contrato'].value_counts(dropna=False)
```

tipo_de_contrato	# count
PRESTACIÓN DE SERVICIOS	1436962
OTRO	64342
DECRETO 092 DE 2017	50144
SUMINISTROS	34535
COMPRAVENTA	27974
OBRA	19571
SUMINISTRO	11339
OTRO TIPO DE CONTRATO	10583
ARRENDAMIENTO DE INMUEBLES	7991
CONSULTORÍA	4136

Creación de banderas de apoyo.

Lo siguiente que hice fue crear una serie de **banderas de validación (flags)**, los cuales permitieron evaluar distintos aspectos clave de los registros. Estas banderas se diseñaron para identificar, entre otros puntos, la existencia de registros duplicados, la validez de los valores monetarios, la presencia de campos clave, y la coherencia entre fechas y estados del contrato.

Adicionalmente, se verificó que las fechas siguieran un orden lógico dentro del ciclo de vida del contrato, por ejemplo, que la fecha de inicio no fuera anterior a la fecha de firma, y que la fecha de finalización no fuera anterior a la fecha de inicio. También se evaluó la relación entre el estado del proceso y la presencia o ausencia de ciertas fechas, entendiendo que algunos estados admiten valores nulos mientras que otros no.

Cada una de estas reglas generó una bandera individual, lo que permitió evaluar cada condición de manera independiente y entender con mayor claridad qué tipo de inconsistencias estaban presentes en los datos.

Finalmente, todas estas banderas fueron **combinadas en un único indicador general**, denominado *flag_registro_valido*. Este indicador permitió clasificar los registros entre válidos y no válidos, facilitando la identificación de aquellos que cumplían con todas las condiciones definidas y aquellos que requerían revisión o análisis adicional.

```
dataCruda['flag_registro_valido'] = (
    dataCruda['flag_valor_valido'] &
    dataCruda['flag_fecha_firma_valida'] &
    dataCruda['flag_inicio_posterior_firma'] &
    dataCruda['flag_fin_posterior_inicio'] &
    dataCruda['flag_claves_presentes'] &
    dataCruda['flag_inicio_coherente_estado'] &
    ~dataCruda['duplicado_tecnico']
)

dataCruda['flag_registro_valido'].value_counts(normalize=True) * 100
```

flag_registro_valido	#	proportion
True	99.15940774587814	
False	0.8405922541218547	

Este enfoque permitió tener un control claro sobre la calidad de la información y dejó los datos preparados para realizar análisis descriptivos y detectar anomalías de forma más confiable.

Creación del quality_log.

Una vez definidos los criterios que permitían identificar qué registros eran válidos y cuáles no, procedí a crear un Quality Log, con el objetivo de clasificar y documentar los registros irregulares encontrados durante el proceso de validación.

El Quality Log se diseñó como una estructura independiente del DataFrame principal, en la cual se registra, para cada fila que no cumple alguna de las reglas definidas, una descripción clara del motivo por el cual el registro no fue aceptado.

df_quality_log['severidad'].value_counts()		df_quality_log['regla'].value_counts()	
✓ 0.0s		✓ 0.0s	
severidad	# count	regla	# count
LEVE	181423	INICIO_INUSUAL_PARA_ESTADO	181423
CRITICA	777	FIN_ANTES_DE_INICIO	465
		INICIO_ANTES_DE_FIRMA	312

Como se observa en la imagen, los datos identificados como inválidos se clasificaron en dos niveles de severidad: **CRÍTICA** y **LEVE**. Esta clasificación permitió tomar decisiones diferenciadas sobre cómo tratar cada tipo de registro durante el procesamiento de la información.

Los registros con severidad **CRÍTICA** corresponden a casos en los que la información presenta errores graves o inconsistencias que afectan directamente la confiabilidad de los datos. Este tipo de errores involucra información sensible que está incorrecta, por lo que estos registros deben ser eliminados del DataFrame procesado. Por otro lado, los registros con severidad **LEVE** representan situaciones inusuales o poco comunes, pero que no afectan de manera directa el análisis de los datos. Un ejemplo de este tipo de casos es la presencia de fechas de inicio en estados previos a la ejecución del contrato. Aunque este comportamiento no es el esperado en la mayoría de los casos, no invalida el registro.

```
motor = getEngine()
cargarDatosProcesados(df_quality_log, motor, 'data_quality_log')
```

Una vez generado el Quality Log, este fue almacenado en la base de datos dentro del esquema de procesados, conservando el detalle de los registros irregulares, su motivo y nivel de severidad. Esto permitió mantener trazabilidad sobre los datos descartados o marcados como alertas.

Posteriormente, se construyó un nuevo DataFrame que contiene únicamente los registros válidos, excluyendo aquellos con errores de severidad CRÍTICA. Las banderas utilizadas durante la validación fueron eliminadas, ya que cumplieron su función como apoyo al control de calidad.

```
motor = getEngine()
cargarDatosProcesados(df_processed, motor, 'secop_contratos_2025_procesados_historico')
```

Finalmente, la información depurada fue cargada en la base de datos bajo la tabla `secop_contratos_2025_procesados_historico`, garantizando que los datos almacenados cumplen con los criterios de calidad definidos y están listos para el análisis.

Separación de datos históricos vs financieros.

Durante el análisis identifiqué una situación crítica: un mismo contrato puede aparecer varias veces en los datos, ya que va pasando por diferentes estados a lo largo de su ciclo de vida (borrador, aprobado, en ejecución, modificado, terminado, liquidado, entre otros).

Desde el punto de vista histórico, estos registros son válidos y necesarios, porque permiten entender cómo evoluciona un contrato en el tiempo. Sin embargo, desde el punto de vista financiero, este comportamiento genera un problema importante: el valor del contrato se repite en cada estado, lo que puede llevar a duplicar o incluso triplicar el valor real si se suman todos los registros sin ningún control.

Para evitar este problema, tomé la decisión de separar la información en dos enfoques:

- Histórico: conserva todos los registros y estados del contrato para análisis de trazabilidad y evolución.
- Financiero: conserva un solo registro por contrato, representando su estado más relevante para efectos contables.

En el dataset financiero, definí una prioridad de estados, dando mayor peso a los estados finales como Terminado, Liquidado o Cerrado, ya que reflejan la información más completa y definitiva del contrato. A partir de esta prioridad, se seleccionó un único registro por contrato, asegurando que el valor contabilizado sea el correcto y evitando la sobreestimación de montos.

```
PRIORIDAD_ESTADO = {
  'TERMINADO': 4,
  'LIQUIDADO': 4,
  'CERRADO': 4,

  'MODIFICADO': 3,
  'CEDIDO': 3,
  'SUSPENDIDO': 3,

  'EN EJECUCIÓN': 2,

  'ACTIVO': 1,
  'APROBADO': 1,
  'BORRADOR': 1,
  'EN APROBACIÓN': 1,
  'CONVOCADO': 1,
  'ADJUDICADO': 1,
  'ENVIADO PROVEEDOR': 1
}

CLAVE_CONTRATO = [
  'numero_del_contrato',
  'documento_proveedor',
  'codigo_entidad_en_secop'
]
```

Para realizar esta separación sin perder información, fue necesario definir una clave de identificación del contrato. Esta clave se construyó a partir del número del contrato, el documento del proveedor y el código de la entidad en SECOP. Esta combinación permite identificar de forma única cada contrato, incluso cuando un mismo número de contrato puede aparecer asociado a diferentes entidades o proveedores. Es importante resaltar que no se elimina información arbitrariamente. Un mismo contrato puede tener múltiples registros debido a los cambios de estado a lo largo del tiempo, y todos estos registros se conservan en el conjunto histórico. En el conjunto financiero, lo que se hace es seleccionar el registro más representativo, según la prioridad de estado y otros criterios temporales, asegurando que el valor contabilizado corresponda a la versión correcta del contrato.

Gracias a esta separación, fue posible garantizar consistencia financiera, mantener la trazabilidad histórica y evitar errores graves en los cálculos de valores contratados.

Una vez seleccionado el registro financiero más representativo por contrato, se excluyeron las columnas auxiliares utilizadas durante el proceso, como la duración del contrato y la prioridad del estado, ya que solo eran necesarias para la selección del registro correcto.

```
columnas_a_excluir = [
    ... 'duracion_dias',
    ... 'prioridad'
]

df_processed = df_processed_financiero.drop(columns=columnas_a_excluir, errors='ignore')

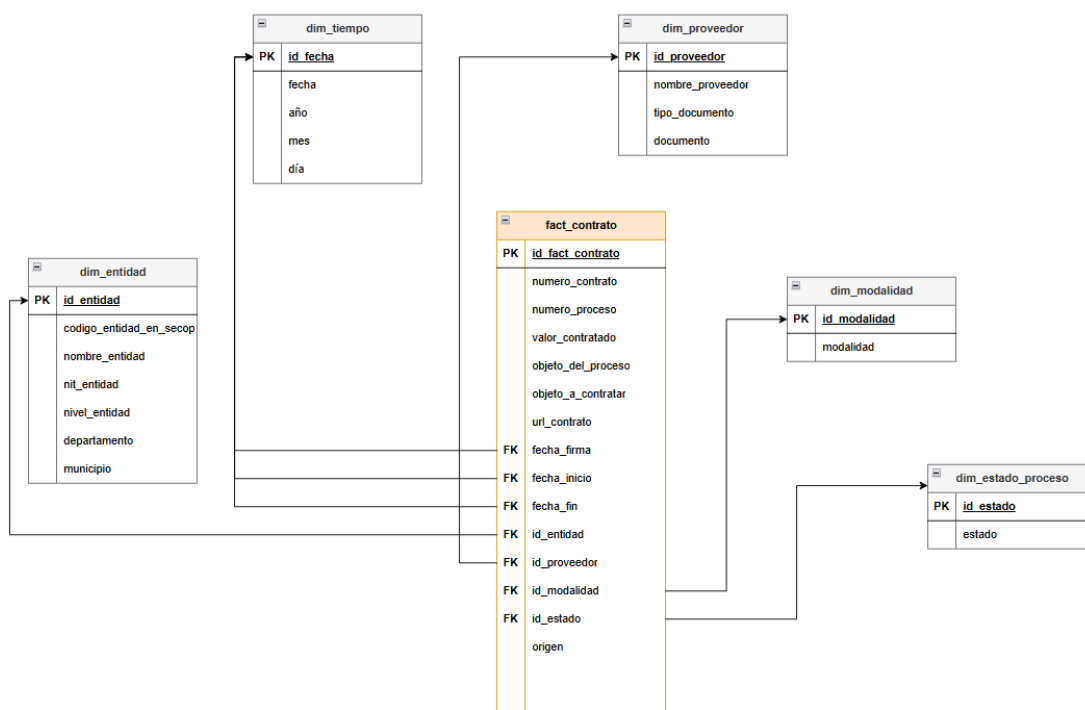
motor = getEngine()

cargarDatosProcesados(df_processed_financiero, motor, 'secop_contratos_2025_procesados_financiero')
```

Finalmente, la información financiera depurada fue almacenada en la base de datos en la tabla secop_contratos_2025_procesados_financiero, quedando lista para su uso en análisis financieros y detección de anomalías sin riesgo de duplicar valores.

Normalización.

Una vez definida la información procesada, se realizó la normalización del modelo de datos con el objetivo de organizar la información de forma clara, evitar duplicidades y facilitar el análisis.



Para esto, se separaron los datos en tablas de dimensiones y una tabla de hechos. Las dimensiones contienen información descriptiva que se repite en muchos registros, como entidades, proveedores, fechas, modalidades y estados del proceso. Al almacenarlas en tablas independientes, se evita repetir la misma información una y otra vez en cada contrato.

La tabla `fact_contrato` actúa como el eje central del modelo y almacena la información principal del contrato, como el número de contrato, número de proceso, valor contratado y objetos del proceso. Esta tabla se relaciona con las dimensiones mediante claves foráneas, lo que permite mantener la consistencia de los datos y facilita las consultas analíticas.

Código SQL.

Como parte de los entregables, se anexan dos archivos en formato `.txt` que contienen el código SQL necesario para realizar la normalización del modelo de datos a partir de las tablas procesadas.

- Un archivo corresponde a la normalización de la tabla **`procesados.secop_contratos_2025_procesados_financiero`**, donde se construye el modelo orientado al análisis financiero

`secop_financiero.txt`

- El segundo archivo corresponde a la normalización de la tabla **`procesados.secop_contratos_2025_procesados_historico`**, enfocada en conservar la trazabilidad completa de los estados del contrato

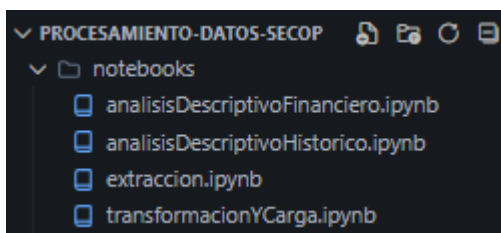
`secop_historico.txt`

En ambos casos, el código SQL crea las tablas de dimensiones y la tabla de hechos, y realiza la carga de información respetando las relaciones definidas en el modelo normalizado. Estos scripts permiten reproducir el proceso completo de normalización directamente desde las tablas procesadas.

Análisis descriptivo.

El análisis descriptivo se desarrolló a partir de dos notebooks principales:

`analisisDescriptivoHistorico.ipynb` y **`analisisDescriptivoFinanciero.ipynb`**, donde se exploraron los datos procesados desde dos enfoques distintos. En esta sección se presentan las conclusiones más relevantes, comenzando por el análisis del conjunto histórico, el cual conserva todos los registros y estados por los que ha pasado cada contrato.



Conclusiones del análisis descriptivo histórico.

- El análisis histórico muestra que para el año 2025 se cuenta con 1.665.766 registros, correspondientes a 1.157.865 contratos únicos y 825.201 procesos únicos. Esto confirma que un mismo contrato puede aparecer varias veces en el dataset debido a los cambios de estado a lo largo de su ciclo de vida.
- En promedio, cada contrato presenta 1,43 registros, lo que indica que la mayoría de los contratos solo tiene un estado registrado, pero existe un subconjunto relevante que pasa por múltiples estados.
- La distribución de estados evidencia que los contratos se concentran principalmente en estados como En ejecución, Modificado y Activo.
- En cuanto a las modalidades de contratación, se observa un claro predominio de la Contratación Directa, seguida por Régimen Especial y Mínima Cuantía, lo que sugiere que gran parte de la contratación pública se realiza bajo esquemas ágiles.
- El análisis por entidades muestra que algunas instituciones concentran un alto volumen de registros. Sin embargo, al observar las entidades con más contratos únicos, el orden cambia, lo que confirma que el volumen de registros históricos no siempre representa un mayor número de contratos, sino una mayor cantidad de cambios de estado.

Conclusiones del análisis descriptivo financiero.

- El análisis financiero se realizó sobre un conjunto de 1.170.546 contratos únicos, lo que confirma que el proceso para eliminar duplicados fue efectivo y permitió trabajar con valores reales sin sobreestimar el gasto público.
- El valor total contratado para el año 2025 asciende a una cifra considerable, evidenciando una alta concentración del gasto en un número reducido de contratos. Esto se refleja en la distribución de los valores, donde la mediana es significativamente menor que el promedio, lo que indica una distribución asimétrica, dominada por contratos de alto valor.
- El análisis por percentiles confirma esta concentración: mientras el 75 % de los contratos tiene valores relativamente bajos, un pequeño porcentaje concentra montos extremadamente altos, lo cual resulta relevante para la identificación de anomalías y focos de atención.
- En cuanto a las entidades, se observa que algunas concentran tanto el mayor número de contratos como los mayores montos contratados, destacándose entidades de nivel nacional y grandes distritos.
- Respecto a las modalidades, la Contratación Directa domina ampliamente el escenario financiero, seguida por el Régimen Especial y la Mínima Cuantía, lo que confirma patrones observados en el análisis histórico.

- El análisis por estado muestra que los mayores valores contratados se concentran en estados como Modificado, En Ejecución y Aprobado, lo que refuerza la importancia de considerar el estado del contrato al momento de analizar el impacto financiero.

Anomalías año 2025.

La identificación de anomalías se realizó teniendo en cuenta tres fuentes principales: el análisis financiero, el análisis histórico y los registros documentados en el Quality Log. Este enfoque permitió detectar comportamientos inusuales sin confundirlos con errores de datos.

ANOMALIA: Contratos con demasiados cambios de estado

```

CLAVE_CONTRATO = [
    'numero_contrato',
    'documento',
    'codigo_entidad_en_secop'
]

cambios_estado = (
    secop_historico
    .groupby(CLAVE_CONTRATO)
    .size()
)

cambios_estado[cambios_estado > cambios_estado.quantile(0.99)]

```

58s

numero_contrato', 'doc...	#
('25-22-105118', '890900286', '20515')	15
('COI.PCCNTR.7124184', '901114890')	6
('COI.PCCNTR.7140570', '7162508', '')	9
('COI.PCCNTR.7140666', '24176583', '')	9
('COI.PCCNTR.7140956', '19272009', '')	9
('COI.PCCNTR.7141445', '103881060', '')	9
('COI.PCCNTR.7141449', '101544716', '')	9
('COI.PCCNTR.7141524', '52560591', '')	9
('COI.PCCNTR.7141730', '102083877', '')	9
('COI.PCCNTR.7141764', '102073708', '')	9

9,315 rows x 1 cols 10 per page

ANOMALIA: Concentración anómala en proveedores

```

concentracion_proveedores = (
    secop_financiero
    .groupby('nombre_proveedor')['valor_contratado']
    .sum()
    .sort_values(ascending=False)
)

concentracion_proveedores.head(10)

```

2.1s Open 'concentracion_proveedores' in Data Wrangler

nombre proveedor	# valor contratado
GECELCA S.A. E.S.P.	4205027751839.0
ZONA FRANCA BARRANQUILLA	2846224257835.0
YESID AVILA TORRES	2386780055500.0
UNION TEMPORAL INTEGRACION LOGISTICA ELECTORAL 2026	2222283357704.0
BANCOLOMBIA	1802611170427.0
HOSPITAL UNIVERSITARIO SAN JUAN DE DIOS Y MATERNO INFANTIL	1615266971115.0
IMPRESA NACIONAL CASA DA MONEDA S.A DE PORTUGAL	1308034931250.0
EMPRESA DE DESARROLLO URBANO DE MEDELLIN	1284258973999.0
EMPRESA DISTRITAL DE DESARROLLO Y RENOVACIÓN URBANO SOSTENIBLE DE SANTA MARTA- EDUS	1025937459727.0
CONSORCIO K&G SAMUEL MEJIA	998049859557.0

10 rows x 1 cols 10 per page

Desde el análisis financiero, se identificó una concentración anómala del valor contratado en un grupo reducido de proveedores. Un número muy pequeño de proveedores concentra montos significativamente altos frente al resto, lo que no implica necesariamente una irregularidad, pero sí representa un comportamiento atípico.

Desde el análisis histórico, se detectaron contratos con un número inusualmente alto de cambios de estado. Estos contratos presentan una dinámica administrativa más compleja que el promedio, lo cual puede indicar procesos con frecuentes modificaciones, ajustes o suspensiones, y por tanto requieren una revisión más detallada.

Adicionalmente, el data_quality_log aportó un conjunto de anomalías de tipo operativo. En particular, los registros clasificados con severidad LEVE fueron considerados como anomalías, ya que representan comportamientos poco comunes —como fechas de inicio presentes en estados previos a la ejecución— que no invalidan el contrato, pero sí se salen del patrón esperado.

En conjunto, las anomalías identificadas no corresponden exclusivamente a errores en los datos, sino a patrones atípicos en la gestión, evolución y concentración financiera de los contratos. Estas situaciones fueron detectadas gracias a la separación entre datos históricos y financieros, así como al control de calidad aplicado durante todo el proceso, permitiendo un análisis más preciso y confiable de la contratación pública en 2025.

Conclusiones.

El desarrollo de este ejercicio fue una experiencia de aprendizaje muy valiosa, principalmente por la complejidad y el volumen del dataset trabajado. La información del SECOP es amplia y diversa, y refleja procesos reales que no siempre siguen un camino claro o lineal. Esto obligó a detenerse, explorar los datos con calma y tomar decisiones con criterio antes de llegar a cualquier conclusión.

Uno de los aprendizajes más importantes fue entender que trabajar con datos reales va mucho más allá de limpiar valores nulos o corregir formatos. Fue necesario comprender el contexto de la contratación pública, los diferentes estados por los que pasa un contrato y cómo estos influyen directamente tanto en el análisis histórico como en el financiero. Sin este entendimiento, los resultados habrían sido fácilmente engañosos.

Además, aplicar reglas de negocio para priorizar y eliminar duplicados, eligiendo estados más representativo de cada contrato, resultó clave para evitar errores y obtener cifras más confiables.

En conclusión, aunque se trató de un dataset exigente de analizar, se logró construir un proceso ordenado, claro y reproducible. Este ejercicio no solo fortaleció habilidades técnicas, sino también la capacidad de análisis y criterio, demostrando que el verdadero valor de los datos está en saber interpretarlos correctamente dentro de su contexto.